1 Exercise 3: Elevator (Arduino Mega)

1.1 Useful References

- ATmega2460 Datasheet
- AVR LIBC manual

1.2 Overview

Using Arduino MEGA, make an elevator using sequential programming and a state machine. The elevator has 5 floors.

1.2.1 Building Circuit

Build the circuit using these components:

- Arduino Mega (Mega 2560)
- 7 x LEDs
- 5 x Switch buttons
- 12 x 220 Ohm resistor
- Wires

The schematic of the circuit is as follows.

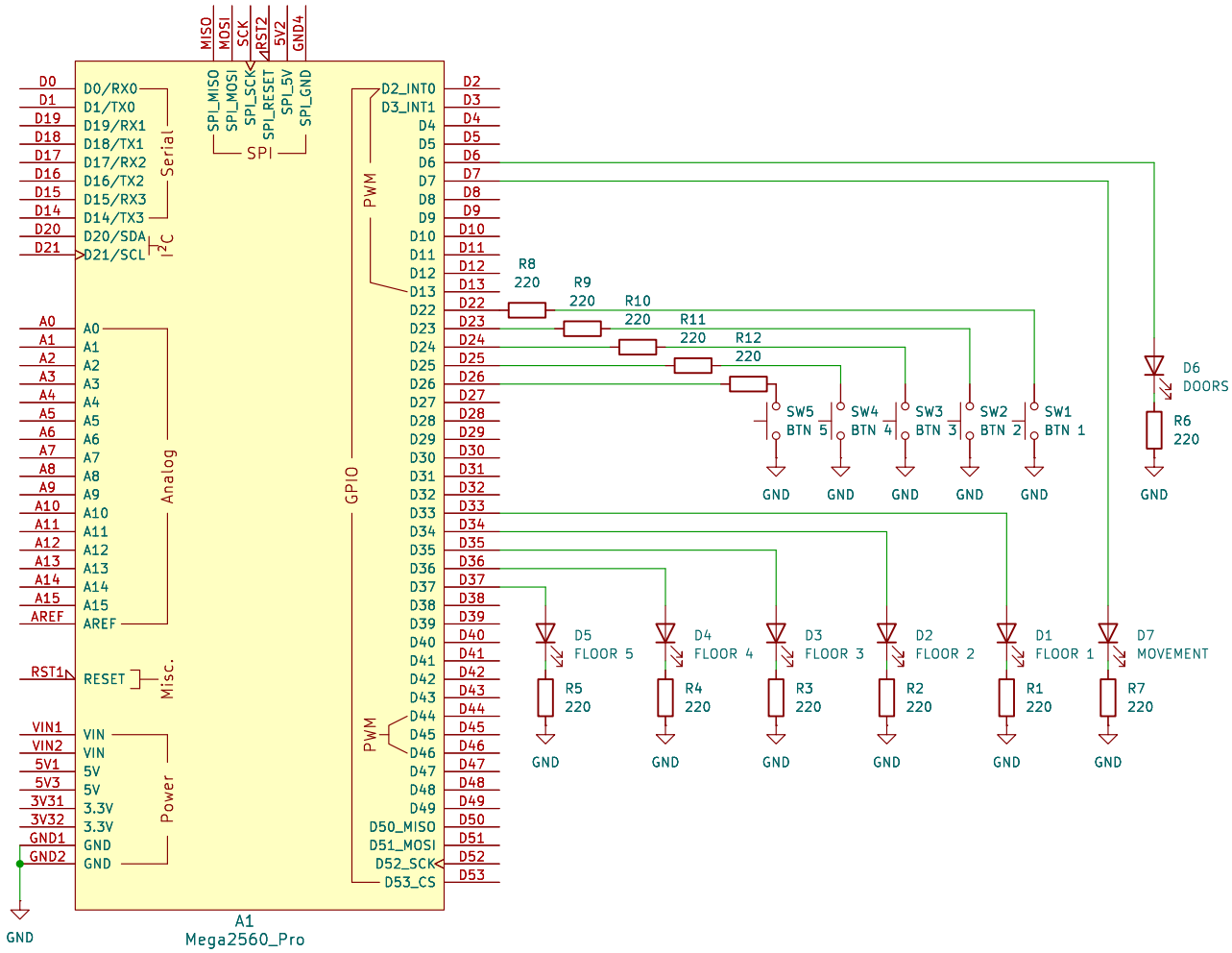

Figure 1: Circuit schematic.

- Buttons (Floors): Connect one leg to ground via a 220 Ω resistor, and the other leg to pin 22 of Arduino MEGA. Set the same connection for other buttons to respective pins 23, 24, 25, and 26. These buttons will be using AVR's internal pull up resistors while configured as input. The purpose of the resistor is to protect pin from short curcuit in case of incorrect programming (E.g. wrong firmware).
- LEDs (Floors): Connect negative leg to ground via a 220 Ω resistor, and the positive leg to pin 33 of Arduino MEGA. Set the same connection for other LEDs to respective pins 34, 35, 36, and 37.
- LED (Door opening): Connect negative leg to ground via a 220 Ω resistor, and the positive leg to pin 6 of Arduino MEGA.
- LED (Movement): Connect negative leg to ground via a 220 Ω resistor, and the positive leg to pin 7 of Arduino MEGA.
- Connect any ground pin of Mega board to your circuit's ground bar.

1.3 State Machine

Elevator states are `IDLE`, `GOINGUP`, `GOINGDOWN`, `DOOR_OPENING`, `DOOR_CLOSING`, and `FAULT`

- The `GOINGUP` and `GOINGDOWN` states move the elevator between floors
- When the elevator reaches the requested floor, it will go to state `DOOR_OPENING`, where it will keep the doors open, for example for 3 seconds. •
- If the user presses the button which is outside of the range of permitted states, the state machine will move to state FAULT, where it will reset itself and move to Idle state.
- Idle state waits for user request for the next floor to go to. When the requested floor is in valid range and not the same as current floor the selection is locked and the state machine transitions to `DOOR_CLOSING` state.
- Finally when the state machine is in `DOOR_CLOSING` it will make a decision of going up or down to the requested floor and transition to the corresponding state.

1.4 Re-using existing code

Add to project the following provided files:

- mcu.h for microcontroller-related defines
- uart.h and uart.c for serial port driver implemented in Exercise 1
- `bit_ops.h` helper header implemented in Exercise 2
- `board_config_2560.h` Arduino Mega board definitions for this exercise.
- `pin_config.h` Header with circuit layout definitions for this exercise. Review this header to find location of `MIN_FLOOR` and `MAX_FLOOR` defines

1.5 Advanced GPIO abstraction layer

Create and add to your project a file named `avr_gpio.h` This file will implement GPIO operations using information about individual IO pins packed into C structures. Start by adding header guard defines and the type definition for the structure we will be using to combine all data about the single IO pin. Note that this code uses `__typeof__` preprocessor function to define pointers to the correct types as described by AVR IO headers.

```c
#ifndef AVR_GPIO_H
#define AVR_GPIO_H

#include <stdio.h>
#include <avr/io.h>
#include "bit_ops.h"

typedef struct avr_gpio_tag {
    __typeof__(PORTA) *port;
    __typeof__(DDRA) *direction;
    __typeof__(PA0) pin_offset;
    __typeof__(PINA) *input;
} avr_gpio_t;

// Add static functions here.

#endif //GPIO_H
```

Now let's define several static functions that will act as helpers operating on the GPIO pins. First we define the 3 functions, that match 3 possible operation states of AVR GPIO pin: output, input without a pull-up resistor (floating) and input with a pull-up resistor. Note that we now use indirect addressing of port/direction and input (pin) registers.

```c
static void set_as_output(avr_gpio_t * gpio_pin) {
    // Output
    SET_BIT(*(gpio_pin->direction), gpio_pin->pin_offset);
}

static void set_as_input_floating(avr_gpio_t * gpio_pin) {
    // Input
    CLEAR_BIT(*(gpio_pin->direction), gpio_pin->pin_offset);
    // No pull-up
    CLEAR_BIT(*(gpio_pin->port), gpio_pin->pin_offset);
}

static void set_as_input_pull_up(avr_gpio_t * gpio_pin) {
    // Input
    CLEAR_BIT(*(gpio_pin->direction), gpio_pin->pin_offset);
    // With pull-up
    SET_BIT(*(gpio_pin->port), gpio_pin->pin_offset);
}
```

Then in a similar manner we add helpers to set and clear pins in output register. Note that if the pin is configured as input these will also toggle pull-up register on and off.

```c
static void set_gpio(avr_gpio_t * gpio_pin) {
    SET_BIT(*(gpio_pin->port), gpio_pin->pin_offset);
}

static void clear_gpio(avr_gpio_t * gpio_pin) {
    CLEAR_BIT(*(gpio_pin->port), gpio_pin->pin_offset);
}
```

And finally we implement a bit reading helper that also uses standard input to log the value we read from pin for debugging. This functionality requires the UART driver implemented in Exercise 1.

```c
static uint8_t read_gpio(avr_gpio_t * gpio_pin) {
    uint8_t result = READ_BIT(*(gpio_pin->input), gpio_pin->pin_offset);
    printf("reading gpio pin input %d result %d \r\n", gpio_pin->pin_offset, result);
    return result;
}
```

1.6 Pin config unit

Create and add to project `pin_config.c` file. Here we will create a runtime initializer that will prepare structures for AVR GPIO driver that we can later use to operate leds and buttons as single value or iterate in array. Add more definitions for floot buttons and leds.

```
#include "pin_config.h"
avr_gpio_t floor_buttons[MAX_FLOOR-MIN_FLOOR+1];
avr_gpio_t floor_leds[MAX_FLOOR-MIN_FLOOR+1];
avr_gpio_t doors_led;
avr_gpio_t movement_led;

void init_avr_gpio_pins(void) {
    floor_buttons[1-MIN_FLOOR].port = &BUTTON_FLOOR_1_PORT;
    floor_buttons[1-MIN_FLOOR].input = &BUTTON_FLOOR_1_INPUT;
    floor_buttons[1-MIN_FLOOR].direction = &BUTTON_FLOOR_1_DIRECTION;
    floor_buttons[1-MIN_FLOOR].pin_offset = BUTTON_FLOOR_1_PIN;

// Add more floor buttons here

    floor_leds[1-MIN_FLOOR].port = &LED_FLOOR_1_PORT;
    floor_leds[1-MIN_FLOOR].input = &LED_FLOOR_1_INPUT;
    floor_leds[1-MIN_FLOOR].direction = &LED_FLOOR_1_DIRECTION;
    floor_leds[1-MIN_FLOOR].pin_offset = LED_FLOOR_1_PIN;

// Add more floor LEDs here

    doors_led.port = &LED_DOORS_PORT;
    doors_led.input = &LED_DOORS_INPUT;
    doors_led.direction = &LED_DOORS_DIRECTION;
    doors_led.pin_offset = LED_DOORS_PIN;

    movement_led.port = &LED_MOVEMENT_PORT;
    movement_led.input = &LED_MOVEMENT_INPUT;
    movement_led.direction = &LED_MOVEMENT_DIRECTION;
    movement_led.pin_offset = LED_MOVEMENT_PIN;
}
```

1.7 main.c

Create and add to project empty main.c file.

First we need to include the configuration headers and our UART and GPIO libraries.

```
#include "mcu.h"
#include <stdint.h>
#include "avr_gpio.h"
#include <util/delay.h>

#include "uart.h"
#include "pin_config.h"
```

Then we define some useful constants defining timing behavior of our state machine.

```
#define DOOR_CLOSE_OPEN_DURATION_MS (3000)
#define FLOOR_MOVING_SPEED_MS (3000)
#define FAULT_BLINK_PERIOD_MS (200)
#define FAULT_BLINK_DURATION_MS (3000)
```

Let's also define a convenient to use enumerable (enum) data type that will represent our state machine's state. Note that it is functionally an integer value but defining it in this way helps compiler and IDEs to provide diagnostics for cases where some state might be left undefined. Explicitly providing values here is optional but will help reading debug logs where states are provided as integer values.

```
typedef enum {
        IDLE = 0,
        GOINGUP = 1,
        GOINGDOWN = 2,
        DOOR_OPENING = 3,
        DOOR_CLOSING = 4,
        FAULT = 5,
} state_t;
```

1.7.1 Interfacing with buttons and LEDs

Write a function that reads button states and returns the pushed button. If several buttons are pressed, the lowest floor is returned. If no buttons are pressed it returns `MIN_FLOOR` - 1. Note: The button pins are pulled up and pushed states corresponds to pin reading 0

```
static int8_t floor_button_choice(void)
{
        for (int8_t i = MIN_FLOOR; i <= MAX_FLOOR; i++) {
                if ( !read_gpio(&floor_buttons[i-MIN_FLOOR]) )
                {
                        return i;
                }
        }
        return MIN_FLOOR-1;
}
```

Add two functions: `floor_led_on` and `floor_led_off` that will switch floor LEDs on and off using gpio set/clear and `floor_leds` array. Add debug logging of these events to help you diagnose circuit connection issues.

```
static void floor_led_on(const int8_t current_floor)
{
...
        set_gpio(&floor_leds[current_floor-MIN_FLOOR]);
}

static void floor_led_off(const int8_t current_floor)
{
...
}
```

1.7.2 State transition condition example

The transition between states in state machine happens when certain conditions are met. One way is to define thesea as a series of rules that take strict precedence. For example a function for `IDLE` state transitions can be defined as follows.

```
state_t idle_state_transition_check(const int8_t requested_floor, const int8_t current_floor) {
        // Keep state when no buttons are pressed
        if (requested_floor == (MIN_FLOOR-1)) {
                return IDLE;
        }

        // If invalid input -> go to FAULT;
        if ((requested_floor > MAX_FLOOR) || (requested_floor < MIN_FLOOR))
        {
                return FAULT;
        }
```

```
        if (requested_floor != current_floor) {
                // If valid movement request -> go to DOOR_CLOSING;
                return DOOR_CLOSING;
        }
        return IDLE;
}
```

Here we first check if any button was pressed. If not we do nothing and do not process other rules. Then we check if the requested floor is invalid. If it is we go to fault state immediately. Finally we check if the elevator needs moving. If yes we start closing the door. As a default outcome we do nothing. For testing your `FAULT` state change the last return `IDLE` to return `FAULT`. This will cause the elevator to go into the fault state when the requested floor is the same floor the elevator is currently at.

1.7.3 State transition actions

Now let's define helpers for things that will happen on state machine transitions.

Firt we define `on_enter` function that instructs actions that occur ONCE when the state machine transitions into the specific state. Here we pass the new state and other context variables and use switch case blocks to handle the logic. For complex actions it is recommended to create individual functions called from here. Note that there are empty case blocks for situations where certain state does not require any action. We still write them explicitly to help with code maintainability when we add or remove new states. Avoid using default case for state machines because it might mask useful compiler diagnostic that usually tells you if you have not defined a state for a switch-case based on enumerable data type.

```
static void on_enter(state_t new_state, int8_t *requested_floor, int8_t *current_floor) {
        printf("on_enter new_state: %d\r\n", new_state);
        switch (new_state) {
                case IDLE:
                        break;
                case GOINGUP:
                        set_gpio(&movement_led); // turn movement LED ON
                        break;
                case GOINGDOWN:
                        set_gpio(&movement_led); // turn movement LED ON
                        break;
                case DOOR_OPENING:
                        set_gpio(&doors_led);
                        _delay_ms(DOOR_CLOSE_OPEN_DURATION_MS);
                        break;
                case DOOR_CLOSING:
                        break;
                case FAULT:
                        // FAULT RECOVERY
                        if (*current_floor < MIN_FLOOR)
                                *current_floor = MIN_FLOOR;
                        if (*current_floor > MAX_FLOOR)
                                *current_floor = MAX_FLOOR;
                        *requested_floor = *current_floor;

                        // Blink movement LED to indicate fault.
                        for (int8_t blink_amount = 0; blink_amount < (FAULT_BLINK_DURATION_MS/FAULT_BLINK
                        {
                                set_gpio(&movement_led);
                                _delay_ms(FAULT_BLINK_PERIOD_MS/2.);
                                clear_gpio(&movement_led);
                                _delay_ms(FAULT_BLINK_PERIOD_MS/2.);
                        }
                        break;
```

```
        }
}
```

Then similarly we define `on_loop` function that instructs actions that occur EVERY iteration of while loop while the state does not changed. Define the following actions:

- In `IDLE` state we wait for 10 ms, then call `floor_button_choice` to get the next requested floor value and store it.
- In `GOINGUP` state we wait for `FLOOR_MOVING_SPEED_MS`, then increment the current floor (note this is a pointer) and set LED on for the ne wvalue of current floor.
- In `GOINGDOWN` state we do the opposite and in reverse. First turn off the led, decrement the counter then wait.

```
static void on_loop(state_t current_state, int8_t *requested_floor, int8_t *current_floor) {
        printf("on_loop current_state: %d\r\n", current_state);
        ...
}
```

Finally define the `on_exit` actions:

- In `GOINGUP` and `GOINGDOWN` state we turn movement LED off.
- In `DOOR_CLOSING` state we wait for `DOOR_CLOSE_OPEN_DURATION_MS` and then turn door LED off.

```
static void on_exit(state_t old_state, int8_t *requested_floor, int8_t *current_floor) {
...
}
```

1.7.4 Main function

First we add boiler plate for serial port initialization.

```
/// There is not much we can do for now. This function will be improved in future.
static void handle_error(uint8_t return_code)
{
        // Non-zero return code indicates critical fault
        if (return_code)
        {
                while(1);
        }
}

int main(void)
{
        // Initialize serial port for standard library
        uint8_t rc = setup_uart_io();
        handle_error(rc);
```

Then we need to configure the GPIO driver and use it to setup the correct GPIO pin settings.

```
        printf("Configuring GPIO mapping.\r\n");
        init_avr_gpio_pins();

        printf("Configuring IO.\r\n");

        for (int8_t i = MIN_FLOOR; i <= MAX_FLOOR; i++) {
```

```
        // Default off
        clear_gpio(&floor_leds[i-MIN_FLOOR]);
        set_as_output(&floor_leds[i-MIN_FLOOR]);
        set_as_input_pull_up(&floor_buttons[i-MIN_FLOOR]);
}

clear_gpio(&movement_led);
set_as_output(&movement_led);
set_gpio(&doors_led);
set_as_output(&doors_led);

printf("Configuring IO done.\r\n");
```

Then we initialize state machine variables and configure pins for `IDLE` state.

```
/* elevator variables, elevator has 5 floors */
volatile state_t elevator_state = IDLE;

int8_t requested_floor = 1;
int8_t current_floor = 1;

set_gpio(&doors_led);
floor_led_on(current_floor);
```

Finally we need to write the main state machine loop. It consists of the two parts. The first part defines the state machine transitions based on external state/input. It is implemented by computing the `next_state` variable. Note that some states share the transition logic, some use dedicated functions, some use simple logic rules and others just map directly into the next state.

```
while (1)
{
        state_t next_state = FAULT;
        /* State machine - switch case */
        switch(elevator_state) {
                case IDLE:
                        next_state = idle_state_transition_check(requested_floor, current_floor)
                break;
                case GOINGUP:
                case GOINGDOWN:
                        next_state = elevator_state;
                        if (requested_floor == current_floor)
                        {
                                //FLOOR REACHED
                                next_state = DOOR_OPENING;
                        }
                break;

                case DOOR_OPENING:
                        next_state = IDLE;
                break;

                case DOOR_CLOSING:
                        if (requested_floor < current_floor)
                        {
                                next_state = GOINGDOWN;
                        }
                        else if (requested_floor > current_floor) {
                                next_state = GOINGUP;
                        } else {
                                next_state = IDLE;
                        }
```

```
                        break;

                        case FAULT:
                                // WRONG FLOOR INPUT. RECOVERY ON ENTER, THEN PROCEED TO IDLE
                                next_state = IDLE;
                        break;
                }
```

The second part logs the important state variables for debugging and implements calling the action handlers. The two possible outcomes of the first part are that a) state does not change this time so we call the loop action and b) that state changes. Then we first call the exit action of current state, update the state and call the enter action of the new state.

```
                printf("elevator_state: %d "
                                "next_state: %d "
                                "requested_floor: %d, "
                                "current_floor: %d\r\n",
                                elevator_state,
                                next_state,
                                requested_floor,
                                current_floor);
                if (elevator_state == next_state) {
                        on_loop(elevator_state, &requested_floor, &current_floor);
                } else {
                        on_exit(elevator_state, &requested_floor, &current_floor);
                        elevator_state = next_state;
                        on_enter(elevator_state, &requested_floor, &current_floor);
                }

        }
}
```